

Research Statement

Emmanouil Giortamis

Technical University of Munich (TUM)

E-Mail: emmanouil.giortamis@tum.de

I am a systems researcher who designs and builds quantum software systems. My work sits at the boundary between quantum hardware and the classical systems that drive it, and it is increasingly organized around a single problem: *making quantum error correction run in real time, at scale.*

Fault-tolerant quantum computing (FTQC) is no longer bottlenecked only by physics. A surface-code machine produces syndrome measurements every roughly one microsecond, and a classical control plane must ingest that stream, discriminate analog readout signals, decode them, and return corrections before dependent operations execute. If the control plane is slower than the measurement rate, the syndrome queue grows without bound and the computation fails, regardless of how good the qubits are. This *backlog problem* converts the central challenge of FTQC into something systems and architecture researchers are uniquely equipped to solve: a hard-real-time, high-throughput, latency-critical pipeline built from heterogeneous accelerators.

1 Vision: The Real-Time QEC Control Plane

Quantum error correction operates as a continuous measure–discriminate–decode–correct cycle. Ancilla qubits are read out, an analog signal is classified into a binary syndrome, a decoder infers the underlying error, and a correction is applied before the next round begins. Every stage of this loop is a classical computation running under a hard deadline, and every stage is currently a bottleneck.

Today these stages are built in isolation. Readout hardware is designed without reference to what the decoder needs; decoders are benchmarked on synthetic syndromes divorced from real readout noise; controllers expose rigid instruction sets that cannot express the branching that dynamic circuits require; and decoder accelerators are statically provisioned, one FPGA image per decoder, even though demand varies by orders of magnitude across program phases and code distances. The result is a pipeline whose end-to-end latency is far worse than the sum of its parts and whose resources are simultaneously over-provisioned and insufficient.

I believe the way out is **hardware-software co-design across the whole loop**, and that the resulting system must be *programmable* rather than hard-wired. The classical control plane of a fault-tolerant machine is, structurally, an operating system problem: it multiplexes scarce accelerator resources among competing consumers, enforces timing contracts, schedules work against deadlines, and must do so while the workload changes underneath it. My background in operating systems, cloud orchestration, and distributed systems is what distinguishes my approach from treating each stage as an isolated hardware-design exercise.

Concretely, I argue for four principles. *First*, co-design across stage boundaries: readout duration, discriminator complexity, and decoder accuracy trade off against each other, and optimizing any one in isolation leaves large gains unrealized. *Second*, latency as a correctness property, not a performance metric: in QEC, missing a deadline is not slow, it is wrong. *Third*, virtualization of decoder resources, so that accelerator capacity can be allocated dynamically rather than frozen into a bitstream. *Fourth*, measured evidence over analytical models: real readout traces, real device topologies, and hardware implementations rather than simulation alone.

To summarize, *my research vision is to architect a programmable, real-time classical control plane for fault-tolerant quantum computing, co-designed from qubit readout through decoding to correction, and built on*

open-source implementations that the community can extend.

2 Research Methodology

Every project I undertake begins with a real bottleneck and proceeds through principled design, careful implementation, and thorough evaluation on both simulators and real hardware. I work with experimentally extracted readout traces, real device topologies and noise profiles, and FPGA implementation reports and RTL simulation rather than analytical estimates alone, because in this domain the difference between a model and a measurement is frequently an order of magnitude.

My research is grounded in modern hardware capabilities and physics-constrained execution. Treating hardware properties as a foundation rather than an inconvenience, I design solutions spanning the full stack, from compilers and formal verification down to microarchitecture and FPGA-based controllers. Performance, scalability, and programmability emerge only when components at these layers are designed in tandem, which is why nearly all of my recent work is explicitly co-design.

I emphasize open science and reproducibility. My major contributions are released as open-source implementations [5, 6, 4, 7, 9, 14, 18, 13, 2, 3, 8], and I consistently pursue artifact-evaluation badges to ensure independent verification. I target high-impact venues at the intersection of systems, architecture, and programming languages, having published at OSDI, SC, MICRO, PLDI, VLDB, HPDC, CGO, SIGMETRICS, and Middleware.

3 Past and Current Research

My work spans the entire real-time QEC loop, together with the broader systems infrastructure that makes it deployable. I group it into four thrusts.

(i) The decoder’s input: readout, discrimination, and classical feedback. Readout is the sole source of ground truth available to a decoder, and it is simultaneously the slowest and most error-prone operation in the stack. In *Oraqle* [7], I built an end-to-end benchmarking framework evaluating qubit-state readout and its impact on QEC across experimentally extracted readout datasets, state-of-the-art ML discriminators, multiple codes, and hardware regimes from current to projected devices. Its findings directly inform control-plane design: measurement duration can be cut substantially with almost no penalty to the logical error rate, while discriminator complexity barely matters because residual error is dominated by device physics, and readout’s influence on logical error rate widens as hardware matures. Readout latency, not discriminator sophistication, is the variable worth optimizing.

MCMit (MICRO’26) [4] attacks the feedback path. Mid-circuit measurements and classical feedback are indispensable for QEC and distributed quantum computing, yet they suffer both high error rates, causing real-time branching errors, and high latencies that amplify decoherence. *MCMit* is a hardware-software co-design addressing both: on the hardware side, a scalable constant-latency multi-control branch instruction and two qubit-state discriminators (a transformer and a CNN) accurate even under short measurement durations; on the software side, static elimination of redundant mid-circuit measurements and stochastic branching to suppress residual errors. Implemented on the QubiC controller and evaluated with experimentally extracted readout traces, the branch instruction reduces feedback latency by up to 70% and improves circuit depth by up to 7 $\times$, while the CNN discriminator achieves up to 62% higher accuracy at short measurement durations, driving 1.2–9.4 $\times$ lower logical error rates. Complementary work on multi-stage Mamba-based readout architectures (QCE’26) [10] pushes discrimination throughput further.

(ii) Real-time decoding and programmable decoder resources. Decoding is where the backlog problem bites hardest. In *Zero-G* [16], we observed that hierarchical decoding is undermined by a mismatch:

pre-decoders produce sparse residual syndromes, but strong decoders are built for dense ones and cannot exploit the sparsity, so the promised speedup never materialises. Zero-G is a stochastic approximate minimum-weight-perfect-matching decoder designed explicitly to sit behind a pre-decoder, dynamically trading latency for accuracy instead of accepting an all-or-nothing choice. By decoupling hardware control from the decoding core, the same implementation deploys across FPGAs and CPUs. Zero-G achieves a $10\times$ latency improvement over state-of-the-art strong decoders at matching accuracy, with worst-case sub-350 ns decoding at code distance $d = 15$, scaling to 400 logical qubits on a single 128-core CPU and 32 logical qubits on an AMD Versal V80 FPGA.

QShell [1] addresses what I consider the central architectural question in this space: decoder resources are currently frozen into FPGA images, one decoder per device, while actual demand varies across program phases, code distances, and decoder families, leaving capacity idle during quiet periods and insufficient during bursts. *QShell* is a co-designed FPGA shell and real-time control-plane operating system providing *virtual decoders: decoder services* that wrap a decoder implementation behind a common interface and a timing contract, so accelerator resources can be allocated and shared independently of the bitstream. The OS verifies communication and processing bounds before admitting syndrome streams, while the shell routes syndromes, schedules decoder access, and returns corrections without per-record host intervention. Evaluated with the Helios and MicroBlossom decoders on U280 and V80 FPGAs, *QShell* fits ten Helios services within 50.9% of V80 LUT capacity at an adapter overhead of 44 ns, roughly 3% of native decoding and interface latency. *QShell* is the clearest expression to date of my thesis that the QEC control plane is an operating-systems problem.

(iii) Fault-tolerant compilation. Real-time decoding is necessary but not sufficient; the programs being corrected must themselves be compiled for fault tolerance. *ECCentric* (SIGMETRICS’26) [18] provides the empirical foundation, characterizing how QEC code performance varies across QPU topologies and noise profiles under realistic hardware assumptions. *Harvest* [11] tackles lattice surgery, where operations that are independent at circuit level still contend for data patches, routing space, and magic states. *Harvest* jointly coordinates circuit-aware placement, congestion-aware routing and scheduling, configurable magic-state availability, and post-scheduling layout pruning, achieving a $4.83\times$ average and up to $17.8\times$ speedup over sequential execution while removing up to 72% of unused magic-state patches. *Chipmunk* [17] targets modular chiplet architectures, which supply the qubit counts FTQC needs but introduce noisy, sparse interconnects that existing compilers cannot handle. Its QEC-aware partitioning preserves logical qubit patch integrity, yielding a $13.5\times$ compilation speedup, $7.5\times$ lower circuit depth, $11.3\times$ fewer SWAPs, and up to two orders of magnitude better logical error rates.

(iv) Quantum operating systems, orchestration, and architecture-level co-design. This thrust is the systems foundation the rest rests on. In *QOS* (OSDI’25) [5] I designed the first holistic operating system for quantum computers, unifying error mitigation, multi-programming, and scheduling in a single stack so that multiple circuits execute concurrently on one QPU without significant fidelity loss; it was the first quantum systems paper accepted at OSDI. *Qonductor* (SC’25) [6], a Best Student Paper Award Finalist, extends this to hybrid quantum-classical workflows with feedback-driven resource estimation and multi-objective scheduling across heterogeneous clusters. *QVM* (PLDI’25) [14] introduced gate virtualization to expose the abstraction of one large QPU over several small ones, and *qTPU* (OSDI’26) [15] extends this into hybrid tensor-network execution that accelerates the classical half of the workload.

In the neutral-atom domain, *Weaver* (CGO’25) [9] was the first retargetable compiler for field-programmable qubit arrays, with a shuttling-aware routing algorithm that co-optimizes gate scheduling and atom movement. *MultiQ* [13] built the first multi-programming environment for neutral-atom QPUs through compiler-controller co-design. *AtomGuard* [12] establishes a formal contract between compilers and heterogeneous neutral-atom hardware via a machine description language, a controller-level ISA, and a sheaf-theoretic verifier; it revealed an average 46% hardware-safety violation rate across three state-of-the-art compilers while verifying two-million-instruction programs in 3.1 seconds. This line of work is why I believe timing

and physical-safety contracts, not just functional correctness, belong in the interface between compiler and controller.

4 Research Agenda: Scalable, Programmable, Real-Time QEC

My group’s central goal will be a classical control plane that keeps pace with fault-tolerant quantum hardware as it scales from tens to thousands of logical qubits. The four directions below are ordered from the decoder outward.

Reconfigurable and programmable decoder architectures. QShell establishes that decoder resources can be virtualized; the next step is making them reconfigurable at runtime. I will investigate decoder architectures that adapt their algorithm, code distance, and accuracy–latency operating point to the current program phase and observed error rate, using partial FPGA reconfiguration and parameterized decoder cores. The research questions are how to express a decoder’s timing contract precisely enough to admit or reject workloads safely, how to migrate an in-flight syndrome stream between decoder instances without dropping deadlines, and how to schedule heterogeneous decoder services under hard real-time constraints. Success means decoding capacity provisioned like cloud capacity rather than like an ASIC.

Scalable real-time decoding on accelerator fabrics. Single-device decoding does not scale to thousands of logical qubits. I will develop distributed decoding architectures that partition syndrome-processing work across fabrics of FPGAs and GPUs while preserving the global consistency that accurate decoding requires, treating this explicitly as a distributed-systems problem with deadlines. Zero-G’s separation of decoding core from hardware control is the right starting point, since it already permits heterogeneous deployment. Key questions are how to partition the decoding graph across devices with bounded communication, how to handle correlated errors spanning partition boundaries, and how to degrade gracefully when a device or link falls behind.

Controller-decoder co-design. MCMit showed that controller instruction-set design directly determines achievable feedback latency. I will pursue this further, co-designing the controller ISA, the discriminator, and the decoder as one system rather than three. This includes instruction sets expressive enough for adaptive, decoder-driven control flow; discriminators placed to minimize end-to-end latency rather than maximize standalone accuracy, which Oraql shows is the wrong objective; and controller architectures that expose timing guarantees to the software above them. The AtomGuard model of a formally verified hardware contract is the template I intend to carry into the superconducting control stack.

Low-latency interconnects for the quantum control plane. The links between accelerators are now a first-order bottleneck. I will design protocols and topologies for nanosecond-scale FPGA–FPGA and FPGA–GPU exchange that organize accelerators into a coherent control plane, so that syndromes, corrections, and control decisions close the loop across many devices. This connects naturally to distributed quantum computing, where remote entanglement operations impose comparable latency demands, and it is where my earlier work on hardware-accelerated replication and consistency in distributed systems [2, 8] becomes directly relevant.

Supporting directions. Two further threads support the above: *QEC-native compilation*, extending Harvest and Chipmunk into an end-to-end compiler that treats logical qubits as first-class citizens and respects the decoder’s real-time budget; and *verification and simulation*, extending AtomGuard’s contracts to distributed quantum systems and simulating the full quantum-classical stack on GPUs, so that control-plane designs can be evaluated before the hardware exists.

References

- [1] Theofilos Augoustis, Peter Wegmann, Anubhav Panda, Emmanouil Giortamis, and Pramod Bhatotia. QShell: An FPGA Shell and Real-time Control-Plane OS for Quantum Error Correction, 2026. Under submission.
- [2] Dimitra Giantsidi, Emmanouil Giortamis, Julian Pritzi, Maurice Bailleu, Manos Kapritsos, and Pramod Bhatotia. Recipe: Hardware-Accelerated Replication Protocols: Rethinking Crash Fault Tolerance Protocols for Untrusted Cloud Environments. In *Proceedings of the 26th International Middleware Conference*, Middleware '25, page 1–15, New York, NY, USA, 2025. Association for Computing Machinery.
- [3] Dimitra Giantsidi, Emmanouil Giortamis, Nathaniel Tornow, Florin Dinu, and Pramod Bhatotia. FlexLog: A Shared Log for Stateful Serverless Computing. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, HPDC '23, page 195–209, New York, NY, USA, 2023. Association for Computing Machinery.
- [4] Emmanouil Giortamis, Felix Gust, Aleksandra Świerkowska, Sandra Stankovic, Yanbin Chen, Innocenzo Fulginiti, Xiaorang Guo, Benjamin Lienhard, Martin Schulz, and Pramod Bhatotia. MCMit: Hardware-Software Co-Design for Mid-Circuit Measurement Error Mitigation. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2026.
- [5] Emmanouil Giortamis, Francisco Romão, Nathaniel Tornow, and Pramod Bhatotia. QOS: Quantum Operating System. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation*, OSDI '25, USA, 2025. USENIX Association.
- [6] Emmanouil Giortamis, Francisco Romão, Nathaniel Tornow, Dmitry Lugovoy, and Pramod Bhatotia. Qonductor: A Cloud Orchestrator for Quantum Computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '25, page 728–745, New York, NY, USA, 2025. Association for Computing Machinery.
- [7] Emmanouil Giortamis, Aleksandra Świerkowska, Sandra Stankovic, Felix Gust, Benjamin Lienhard, and Pramod Bhatotia. Oracle: An Empirical Analysis of Qubit Readout and Discriminators in Quantum Error Correction, 2026.
- [8] Antonios Katsarakis, Emmanouil Giortamis, Vasilis Gavrielatos, Pramod Bhatotia, Aleksandar Dragojevic, Boris Grot, Vijay Nagarajan, and Panagiota Fatourou. The LAW Theorem: Local Reads and Linearizable Asynchronous Replication. *Proceedings of the VLDB Endowment*, 18(9):2831–2845, 2025.
- [9] Oğuzcan Kirmemiş, Francisco Romão, Emmanouil Giortamis, and Pramod Bhatotia. Weaver: A Retargetable Compiler Framework for FPQA Quantum Architectures. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, CGO '25, page 299–316, New York, NY, USA, 2025. Association for Computing Machinery.
- [10] Luca Otting, Xiaorang Guo, Emmanouil Giortamis, Benjamin Lienhard, Pramod Bhatotia, and Martin Schulz. Multi-Stage Mamba-Based Architecture for Fast and Scalable Superconducting Qubit Readout. In *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2026.
- [11] Jannik Pflieger, Aleksandra Świerkowska, Emmanouil Giortamis, and Pramod Bhatotia. Harvest: Resource-Aware Quantum Compilation for Magic State Protocols, 2026.
- [12] Francisco Romão, Nathaniel Tornow, Emmanouil Giortamis, Dennis Sprokholt, and Pramod Bhatotia. Atom-Guard: An Architecture-Centric ISA for Verified Compilation on Neutral Atom Quantum Architectures, 2026. Under submission.
- [13] Francisco Romão, Daniel Vonk, Emmanouil Giortamis, Dennis Sprokholt, and Pramod Bhatotia. MultiQ: Multi-Programming Neutral Atom Quantum Architectures, 2026.
- [14] Nathaniel Tornow, Emmanouil Giortamis, and Pramod Bhatotia. QVM: Quantum Gate Virtualization Machine. *Proc. ACM Program. Lang.*, 9(PLDI), June 2025.
- [15] Nathaniel Tornow, Emmanouil Giortamis, Dennis Sprokholt, Christian B. Mendl, and Pramod Bhatotia. qTPU: Hybrid Tensor Networks for Quantum-Classical Acceleration. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026.
- [16] Peter Wegmann, Theofilos Augoustis, Aleksandra Świerkowska, Emmanouil Giortamis, and Pramod Bhatotia. Zero-G: A Pre-Decoder-Aware Decoder for Quantum Error Correction, 2026.

- [17] Peter Wegmann, Aleksandra Świerkowska, Emmanouil Giortamis, and Pramod Bhatotia. Chipmunq: A Fault-Tolerant Compiler for Chiplet Quantum Architectures, 2026.
- [18] Aleksandra Świerkowska, Jannik Pflieger, Emmanouil Giortamis, and Pramod Bhatotia. ECCentric: An Empirical Analysis of Quantum Error Correction Codes, 2025.